

Planning in Nondeterministic Domains for LTL_f Goals

Game-theoretic Approach to LTL_f Planning and Synthesis

Giuseppe De Giacomo



Planning in Nondeterministic Domain

- **Fluents** $\mathcal{F}$ (propositions) – controlled by the environment
- **Actions** $\mathcal{A}$ (actions) – controlled by the agent
- **Domain** D – specification of the dynamics
- **Goal** G – propositional formula on fluents describing desired state of affairs to be reached

Planning as game between two players

- **Arena**: the domain
- **Players**: **agent** and **environment**
- **Game**: **agent** tries to force eventually reaching G no matter how other **environment** reacts
- **Problem**: find **agent-strategy** $\sigma_a : (2^{\mathcal{F}})^* \rightarrow \mathcal{A}$ to win the game

Complexity

EXPTIME-complete in size of domain specified in PDDL.

Synthesis

- **Inputs** $\mathcal{X}$ (propositions) – controlled by the environment
- **Outputs** $\mathcal{Y}$ (propositions) – controlled by the agent
- **Domain** – **not considered**
- **Goal** φ – arbitrary LTL_f (or other temporal logic specification) on both $\mathcal{X}$ and $\mathcal{Y}$

Synthesis as game between two players

- **Arena**: unconstraint! clique among all possible assignments for $\mathcal{X}$ and $\mathcal{Y}$
- **Players**: **agent** and **environment**
- **Game**: **agent** tries to force a play that satisfies φ no matter how other **environment** reacts.
- **Problem**: find **agent-strategy** $\sigma_a (2^{\mathcal{X}})^* \rightarrow 2^{\mathcal{Y}}$ to win the game.

Complexity

2EXPTIME-complete in size of φ .

Planning in Nondeterministic Domain

- **Fluents** $\mathcal{F}$ (propositions) – controlled by the environment
- **Actions** $\mathcal{A}$ (actions) – controlled by the agent
- **Domain** D – specification of the dynamics
- **Goal** G – propositional formula on fluents describing desired state of affairs to be reached

Planning as game between two players

Arena: the domain

- **Players:** **agent** and **environment**
- **Game:** **agent** tries to force eventually reaching G no matter how other **environment** reacts
- **Problem:** find **agent-strategy** $\sigma_a : (2^{\mathcal{F}})^* \rightarrow \mathcal{A}$ to win the game

Complexity

EXPTIME-complete in size of domain specified in PDDL.

Synthesis

- **Inputs** $\mathcal{X}$ (propositions) – controlled by the environment
- **Outputs** $\mathcal{Y}$ (propositions) – controlled by the agent
- **Domain** – **not considered**
- **Goal** φ – arbitrary LTL_f (or other temporal logic specification) on both $\mathcal{X}$ and $\mathcal{Y}$

Synthesis as game between two players

Arena: unconstrained! clique among all possible assignments for $\mathcal{X}$ and $\mathcal{Y}$

- **Players:** **agent** and **environment**
- **Game:** **agent** tries to force a play that satisfies φ no matter how other **environment** reacts.
- **Problem:** find **agent-strategy** $\sigma_a : (2^{\mathcal{X}})^* \rightarrow 2^{\mathcal{Y}}$ to win the game.

Complexity

2EXPTIME-complete in size of φ .

Planning in Nondeterministic Domain

- **Fluents** $\mathcal{F}$ (propositions) – controlled by the environment
- **Actions** $\mathcal{A}$ (actions) – controlled by the agent
- **Domain** D – specification of the dynamics
- **Goal** G – propositional formula on fluents describing desired state of affairs to be reached

Planning as game between two players

Arena: the domain

- **Players:** **agent** and **environment**
- **Game:** **agent** tries to force eventually reaching G no matter how other **environment** reacts
- **Problem:** find **agent-strategy** $\sigma_a : (2^{\mathcal{F}})^* \rightarrow \mathcal{A}$ to win the game

Complexity

EXPTIME-complete in size of domain specified in PDDL.

Synthesis

- **Inputs** $\mathcal{X}$ (propositions) – controlled by the environment
- **Outputs** $\mathcal{Y}$ (propositions) – controlled by the agent
- **Domain** – **not considered**
- **Goal** φ – arbitrary LTL_f (or other temporal logic specification) on both $\mathcal{X}$ and $\mathcal{Y}$

Synthesis as game between two players

Arena: unconstrained! clique among all possible assignments for $\mathcal{X}$ and $\mathcal{Y}$

- **Players:** **agent** and **environment**
- **Game:** **agent** tries to force a play that satisfies φ no matter how other **environment** reacts.
- **Problem:** find **agent-strategy** $\sigma_a(2^{\mathcal{X}})^* \rightarrow 2^{\mathcal{Y}}$ to win the game.

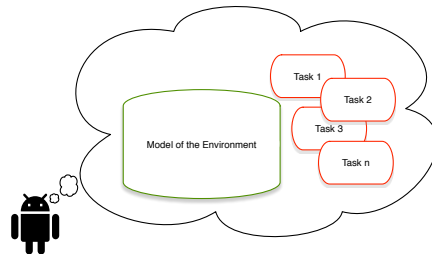
Complexity

2EXPTIME-complete in size of φ .

We want to revisit the assumption that the environment is unconstrained!

Planning in nondeterministic domains

- Environment Model (DOM)
 - ▶ Environment model is called “domain”
 - ▶ Specs of environment's behaviors of the world in response to agent's action
 - ▶ Domain expressed as with specific formalisms
 - ★ PDDL
 - ▶ DOM is, or better generates, a non-deterministic transition system, i.e., a **game arena for two players Agent and Env!**
- Agent Task (GOAL)
 - ▶ Agent task is called “goal”
 - ▶ Specs of task to achieve
 - ▶ GOAL expressed as reaching a state of the domain with desired properties
- Find agent plan/program/strategy/policy that fulfills GOAL in DOM



*Find plan that fulfills the desired task
in spite of how the environment responds,
i.e., wins the GOAL in nondeterministic DOM*

Transition system induced by a nondeterministic domain

A nondeterministic domain $D = (\mathcal{F}, \mathcal{A}, I)$ induces a transition system $T_D = (2^{\mathcal{F}}, \mathcal{A}, s_0, \alpha, \delta)$ where:

- $\mathcal{F}$ is the set of **fluents** (atomic propositions)
- $\mathcal{A}$ is the set of **actions** (atomic symbols)
- $2^{\mathcal{F}}$ is the set of states
- s_0 is the initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a, s')$ with $a \in \alpha(s)$ represents **nondeterministic action effects (including frame)**.

Note the transition function is now a transition relation, i.e., given a state s and an action a we have a set of possible successor states $\{s' \mid \delta(s, a, s')\}$.

Who controls what?

Fluents controlled by **environment**

Actions controlled by **agent**

Observe: $\delta(s, a, s')$

Game arena induced by a nondeterministic domain

If we consider this information on the control, then T_D is in fact a **game arena**: $T_D = (2^{\mathcal{F}}, \mathcal{A}, s_0, \alpha, \delta)$ where:

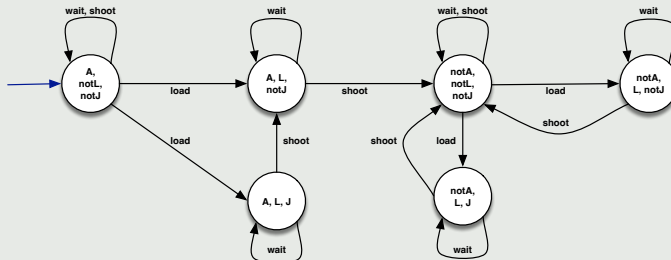
- $\mathcal{F}$ is the set of **fluents** (atomic propositions) - **controlled by the environment**
- $\mathcal{A}$ is the set of **actions** (atomic symbols) - **controlled by agent**
- $2^{\mathcal{F}}$ is the set of states
- s_0 is the initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a, s')$ with $a \in \alpha(s)$ represents **nondeterministic actions effects** (including frame).

Hence to execute a transition in state s

- ▶ The **agent** needs to choose the action a
- ▶ The **environment** need to choose the resulting state s' .

Yale shooting game arena

Yale Shooting domain D induces the following game arena T_D :



Planning

Given a nondeterministic domain D and a goal G in propositional logic:

- Find agent executable strategy (or plan) σ_a such that for every environment strategies σ_e that are compliant with D , we have that the $play(\sigma_a, \sigma_e) = s_0, a_1, s_1, \dots, s_{n-1}, a_n, s_n$ is such that $s_n \models G$.

In other words find a strategy (or plan) σ_a that reaches a state where G holds no matter what the environment does.

The strategy σ_a , if it exists is called **winning strategy**

(1) In order express arbitrary goals in LTL_f –or LTL[¬], we need to:

Represent actions as propositions

Decide how we represent actions as propositions of LTL_f formulas.

- Use one proposition a for each action a . Then:
 - ▶ We need to add the requirement that at most one action proposition is true in each instant $\Box(a \supset \bigwedge_{b \in A \wedge b \neq a} \neg b)$.
- Use a binary (logarithmic) encoding of action each a . Then:
 - ▶ Each action a is represented as a boolean formula a over the propositions for the binary encoding;
 - ▶ Some binary encoding will correspond to non-existing actions, if the number of actions is not a power of 2. In this case we need to specify what these spurious action do in the transition system, e.g., *nope*, or we need to forbid them.

For now, we will adopt the first way of representing actions, but later when we study symbolic technique we will also use the latter.

(cf. LTL_f Model Checking)

(2) In order express arbitrary goals in LTL_f –or LTL–, we also need to:

Pair actions and states in a time instant

Decide how we need to pair actions and states in a time instant

- Pair the agent **action and the resulting state**, (in fact labeling of the state) of the environment
*The propositional representation a for an action a will stand for “**action a just executed**”.*
- Pair current environment (labeling of the) **state and the next action** instructed by the agent
*The propositional representation a for an action a will stand for “**action a just instructed to be executed next**”.*

Both ways of pairing actions and states are fine. But choosing one or the other is essential, because it changes how we specify properties in LTL_f.

LTL_f-traces

A T_D trace $s_0, a_1, s_1, \dots, a_n, s_n$ induces a corresponding LTL_f-trace:

- If we pair **action and the resulting state**: $(dummy, s_0), (a_1, s_1), \dots, (a_n, s_n)$, where *dummy* is a dummy starting action.
- If we pair **state and the next action**: $(s_0, a_1), (s_1, a_2) \dots, (s_{n-1}, a_n), (s_n, dummy)$, where *dummy* is a dummy ending action.

Example

The way we pair actions and states changes how we specify properties in LTL_f:

Suppose we want to say:

every time that ϕ_1 is true in the current state if we do action a we get ϕ_2 in the next state".

- If we pair **action and the resulting state**, we write: $\Box(\phi_1 \supset \bullet(a \supset \phi_2))$
- If we pair **state and the next action**, we write: $\Box((\phi_1 \wedge a) \supset \bullet\phi_2)$

In this course we pair action and the resulting state to have traces that represents cleanly histories (things already happened).

LTL_f-traces

A T_D trace $s_0, a_1, s_1, \dots, a_n, s_n$ induces a corresponding LTL_f-trace:

- If we pair **action and the resulting state**: $(dummy, s_0), (a_1, s_1), \dots, (a_n, s_n)$, where *dummy* is a dummy starting action.
- If we pair **state and the next action**: $(s_0, a_1), (s_1, a_2) \dots, (s_{n-1}, a_n), (s_n, dummy)$, where *dummy* is a dummy ending action.

Example

The way we pair actions and states changes how we specify properties in LTL_f:

Suppose we want to say:

every time that ϕ_1 is true in the current state if we do action a we get ϕ_2 in the next state".

- If we pair **action and the resulting state**, we write: $\Box(\phi_1 \supset \bullet(a \supset \phi_2))$
- If we pair **state and the next action**, we write: $\Box((\phi_1 \wedge a) \supset \bullet\phi_2)$

In this course we pair action and the resulting state to have traces that represents cleanly histories (things already happened).

LTL_f-traces

A T_D trace $s_0, a_1, s_1, \dots, a_n, s_n$ induces a corresponding LTL_f-trace:

- If we pair **action and the resulting state**: $(dummy, s_0), (a_1, s_1), \dots, (a_n, s_n)$, where *dummy* is a dummy starting action.
- If we pair **state and the next action**: $(s_0, a_1), (s_1, a_2) \dots, (s_{n-1}, a_n), (s_n, dummy)$, where *dummy* is a dummy ending action.

Example

The way we pair actions and states changes how we specify properties in LTL_f:

Suppose we want to say:

every time that ϕ_1 is true in the current state if we do action a we get ϕ_2 in the next state".

- If we pair **action and the resulting state**, we write: $\Box(\phi_1 \supset \bullet(a \supset \phi_2))$
- If we pair **state and the next action**, we write: $\Box((\phi_1 \wedge a) \supset \bullet\phi_2)$

In this course we pair action and the resulting state to have traces that represents cleanly histories (things already happened).

With these decisions taken we can transform the nondeterministic domain $T_D = (2^{\mathcal{F}}, \mathcal{A}, s_0, \alpha, \delta)$ into an automaton recognizing all its traces.

Automaton A_D for D is a **DFA!!!**

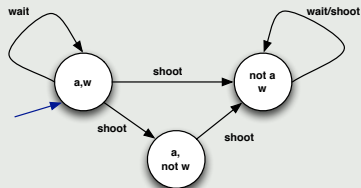
$A_D = (2^{\mathcal{F} \cup \mathcal{A}}, Q, q_{init}, \rho, F)$ where:

- $2^{\mathcal{F} \cup \mathcal{A}}$ alphabet (actions $\mathcal{A}$ include dummy *start* action)
- $Q = 2^{\mathcal{F}} \cup \{q_{init}\}$ set of states
- q_{init} dummy initial state
- $F = 2^{\mathcal{F}}$ (all states of the domain are final)
- $\rho(s, [a, s']) = s'$ **with** $a \in \alpha(s)$, **and** $\delta(s, a, s')$ $\rho(q_{init}, [start, s_0]) = s_0$

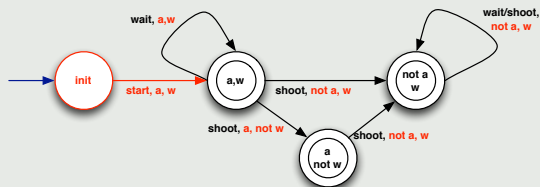
(notation: $[a, s']$ stands for $\{a\} \cup s'$)

Example (Simplified Yale shooting domain variant)

- Domain $\mathcal{T}_D$:



- DFA A_D :



Cartesian Product/Intersection of Automata

Let $A_1 = (\Sigma, S_1, s_1^0, \rho_1, F_1)$ and $A_2 = (\Sigma, S_2, s_2^0, \rho_2, F_2)$. Intuitively, we construct the **intersection (or AND)** $A_1 \times A_2$, also called (synchronous) **Cartesian product**, that runs simultaneously both A_1 and A_2 on the input word and accepts when **both** accept. We define $A_1 \times A_2$ as:

$$A_1 \times A_2 = (\Sigma, S_1 \times S_2, (s_1^0, s_2^0), \rho, F_1 \times F_2)$$

where:

- The **transition function** requires to execute the two automata **concurrently** in a **synchronized** way:

$$((s_1, s_2), a, (s'_1, s'_2)) \in \rho \quad \text{iff} \quad (s_1, a, s'_1) \in \rho_1 \quad \text{and} \quad (s_2, a, s'_2) \in \rho_2$$

- The condition defining the **final states**:

$$F_1 \times F_2$$

says that we accept a word if **both** automata A_1 and A_2 accept it.

It is easy to see that the size of $A_1 \times A_2$ is the product of the sizes of A_1 and of A_2 , and that $\mathcal{L}(A_1 \times A_2) = \mathcal{L}(A_1) \cap \mathcal{L}(A_2)$.

Planning in Nondeterministic Domains for $LT\mathcal{L}_f$ Goals

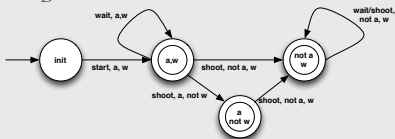
Planning in nondeterministic domains

- Set the **arena** formed by all traces that satisfy both the DFA A_D for D and the DFA for $\Diamond G$ where G is the goal.
- Compute a **winning strategy**.

(EXPTIME-complete in D , constant in G)

Example (Simplified Yale shooting domain)

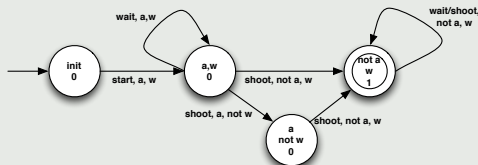
A_D



$A_{\Diamond \neg a}$



$A_D \cap A_{\Diamond \neg a}$:



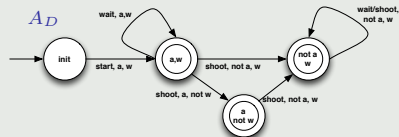
strategy

$init, 0$	$\rightarrow$	$start$
$a, w, 0$	$\rightarrow$	$shoot$
$a, \neg w, 0$	$\rightarrow$	$shoot$
$\neg a, w, 1$	$\rightarrow$	$win!$

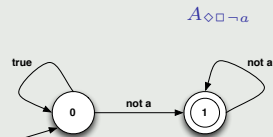
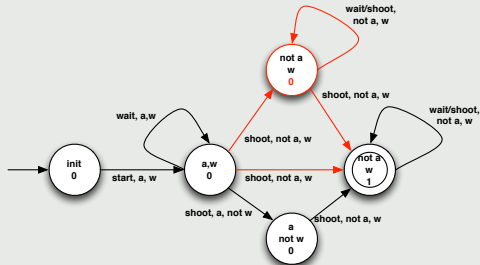
Planning in Nondeterministic Domains for LTL_f Goals

Example (Simplified Yale shooting domain)

Consider the goal $\Diamond \Box \neg a$.



$A_D \cap A_{\Diamond \Box \neg a}$:



Can we use directly NFA's?

No, because of a basic mismatch

- NFA have perfect **foresight**, or **clairvoyance**
- Strategies must be runnable: **depend only on past**, not future

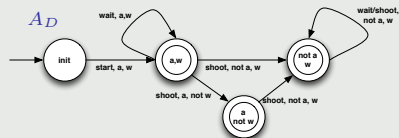
(angelic nondeterminism)

(devilish nondeterminism)

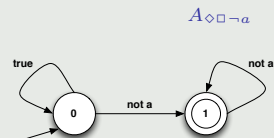
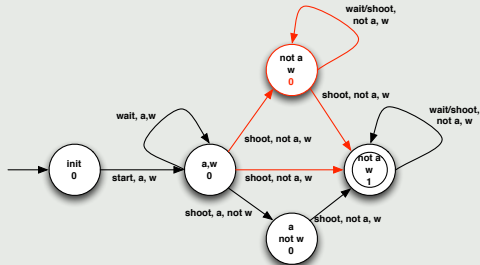
Planning in Nondeterministic Domains for LTL_f Goals

Example (Simplified Yale shooting domain)

Consider the goal $\Diamond \Box \neg a$.



$A_D \cap A_{\Diamond \Box \neg a}$:



Can we use directly NFA's?

No, because of a basic mismatch

- NFA have perfect **foresight**, or **clairvoyance**
- Strategies must be runnable: **depend only on past**, not future

(angelic nondeterminism)

(devilish nondeterminism)

Planning in Nondeterministic Domains for $LT\mathcal{L}_f$ Goals

We need first to determinize the NFA for $LT\mathcal{L}_f$ formula

NFA for $\Diamond\Box\neg a$



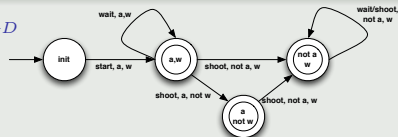
corresponding DFA



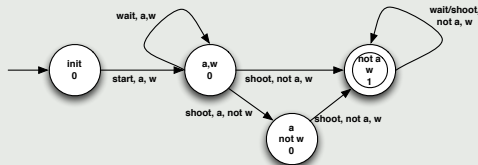
(DFA can be exponential in NFA in general)

Example (Simplified Yale shooting domain)

A_D



$A_D \cap A_{\Diamond\Box\neg a}$:



$A_{\Diamond\Box\neg a}$



strategy

$init, 0$	$\rightarrow$	start
$a, w, 0$	$\rightarrow$	shoot
$a, \neg w, 0$	$\rightarrow$	shoot
$\neg a, w, 1$	$\rightarrow$	win!

DFA games

A **DFA game** $\mathcal{G} = (2^{\mathcal{F} \cup \mathcal{A}}, S, s_{init}, \varrho, F)$, is such that:

- $\mathcal{F}$ controlled by **environment**; $\mathcal{A}$ controlled by **agent**;
- $2^{\mathcal{F} \cup \mathcal{A}}$, alphabet of game;
- S , states of game;
- s_{init} , initial state of game;
- $\varrho : S \times 2^{\mathcal{F} \cup \mathcal{A}} \rightarrow S$, transition function of the game: given current state s and a choice of action a and resulting fluents values e , the resulting state of game is $\varrho(s, [a, e]) = s'$;
- F , final states of game, where game can be considered terminated.

Winning Strategy:

- A play is **winning** for the agent if such a play leads from the initial to a final state.
- A **strategy** for the agent is a function $f : (2^{\mathcal{F}})^* \rightarrow \mathcal{A}$ that, given a **history of choices from the environment**, decides which action $\mathcal{A}$ to do next.
- A **winning strategy** is a strategy $f : (2^{\mathcal{F}})^* \rightarrow \mathcal{A}$ such that for all traces π with $a_i = f(\pi_{\mathcal{F}}|_i)$ we have that π leads to a final state of $\mathcal{G}$.

DFA games

A **DFA game** $\mathcal{G} = (2^{\mathcal{F} \cup \mathcal{A}}, S, s_{init}, \varrho, F)$, is such that:

- $\mathcal{F}$ controlled by **environment**; $\mathcal{A}$ controlled by **agent**;
- $2^{\mathcal{F} \cup \mathcal{A}}$, alphabet of game;
- S , states of game;
- s_{init} , initial state of game;
- $\varrho : S \times 2^{\mathcal{F} \cup \mathcal{A}} \rightarrow S$, transition function of the game: given current state s and a choice of action a and resulting fluents values e , the resulting state of game is $\varrho(s, [a, e]) = s'$;
- F , final states of game, where game can be considered terminated.

Winning Strategy:

- A play is **winning** for the agent if such a play leads from the initial to a final state.
- A **strategy** for the agent is a function $f : (2^{\mathcal{F}})^* \rightarrow \mathcal{A}$ that, given a **history of choices from the environment**, decides which action $\mathcal{A}$ to do next.
- A **winning strategy** is a strategy $f : (2^{\mathcal{F}})^* \rightarrow \mathcal{A}$ such that for all traces π with $a_i = f(\pi_{\mathcal{F}}|_i)$ we have that π leads to a final state of $\mathcal{G}$.

Winning condition for DFA games

Let

$$PreAdv(\mathcal{S}) = \{s \in S \mid \exists a \in \alpha(s). \forall e \in 2^{\mathcal{F}}. \varrho(s, [a, e]) \in S\}$$

Compute the set *Win* of winning states of a DFA game $\mathcal{G}$, i.e., states from which the agent can win the game $\mathcal{G}$, by **least-fixpoint**:

- $Win_0 = F$ (the final states of $\mathcal{G}$)
- $Win_{i+1} = Win_i \cup PreAdv(Win_i)$
- $Win = \bigcup_i Win_i$

*(Computing *Win* is linear in the number of states in $\mathcal{G}$)*

Computing the winning strategy

Let's define $\omega : S \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \mid \text{if } s \in Win_{i+1} - Win_i \text{ then } \forall e. \varrho(s, [a, e]) \in Win_i\}$$

- **Every way** of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy** for $\mathcal{G}$.
- Note s is a **state of the game**! not of the domain only!
To phrase ω wrt the domain only, we need to return a **stateful transducer** with transitions from the game.

Planning in Nondeterministic Domains for LTL_f Goals

Algorithm: $FOND_{sp}$ for LDL_f/LTL_f goals

- 1: Given LTL_f domain D and goal φ
- 2: Compute NFA for φ (exponential)
- 3: Determinize NFA to DFA (exponential)
- 4: Compute intersection with DFA of D (polynomial)
- 5: Synthesize winning strategy for DFA game (linear)
- 6: Return strategy

Theorem

Planning in nondeterministic domains for LTL_f goals is:

- EXPTIME-complete in the domain (*compactly represented using of fluents – polynomial in number of states*);
- 2-EXPTIME-complete in the goal.



Task

Task for the agent, also called *goal*

- It is a *specification of the good traces* (i.e., plays generated by agent and env that are considered good.)
- Typically of a *simpler form* wrt synthesis, i.e.,:

$$\textit{eventually GoodStateOfAffairs} \wedge \textit{always} \neg \textit{PreViolated}$$

- But goal can also be temporally extended, i.e., arbitrary formulas expressed in LTL_f



Domain

- Planning consider the agent acting in a (nondeterministic) domain
- The domain is a model of how the environment works
- That is, it is a specification of the possible environment behaviors, that is:

A specification of how the environment responds to agent actions.

The presence of domain is a crucial point of planning since the beginning!

Planning in nondeterministic domains

Transition system induced by a nondeterministic domain

A nondeterministic domain $D = (\mathcal{F}, \mathcal{A}, I)$ induces a transition system $T_D = (2^{\mathcal{F}}, \mathcal{A}, s_0, \alpha, \delta)$ where:

- $\mathcal{F}$ is the set of **fluents** (atomic propositions)
- $\mathcal{A}$ is the set of **actions** (atomic symbols)
- $2^{\mathcal{F}}$ is the set of states
- s_0 is the initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**

Note: fulfilling action precondition is a responsibility of the agent!

- $\delta(s, a, s')$ with $a \in \alpha(s)$ represents **nondeterministic action effects** (including frame).

Note: fulfilling action effects + frame is a responsibility of the environment!

Remove action preconditions from the domain

In the following we assume action have **no preconditions** in the domains.

- When preconditions are not satisfied the environment remains in the same state. *—requires conditional effects*
- In traces/plays, “agent can select an action only if its satisfies its precondition” can be expressed in the (LTL_f) goal.
- Indeed: if $Pre(a) = \varphi_a$ then it suffices to require in the goal $\Box((\bigcirc a) \supset \varphi_a)$, i.e.:

“Always, if the action a has been just executed next, then its precondition φ_a is now satisfied.”

In this way, the domain becomes a specification of the environment.

Planning in nondeterministic domains

Transition system induced by a nondeterministic domain

A nondeterministic domain $D = (\mathcal{F}, \mathcal{A}, I)$ induces a transition system $T_D = (2^{\mathcal{F}}, \mathcal{A}, s_0, \alpha, \delta)$ where:

- $\mathcal{F}$ is the set of **fluents** (atomic propositions)
- $\mathcal{A}$ is the set of **actions** (atomic symbols)
- $2^{\mathcal{F}}$ is the set of states
- s_0 is the initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**

Note: fulfilling action precondition is a responsibility of the agent!

- $\delta(s, a, s')$ with $a \in \alpha(s)$ represents **nondeterministic action effects** (including frame).

Note: fulfilling action effects + frame is a responsibility of the environment!

Remove action preconditions from the domain

In the following we assume action have **no preconditions** in the domains.

- When preconditions are not satisfied the **environment remains in the same state**. *—requires conditional effects*
- In traces/plays, “**agent can select an action only if its satisfies its precondition**” can be expressed in the (LTL_f) goal.
- Indeed: if $Pre(a) = \varphi_a$ then it suffices to require in the goal $\Box((\bigcirc a) \supset \varphi_a)$, i.e.:

“Always, if the action a has been just executed next, then its precondition φ_a is now satisfied.”

In this way, the domain becomes a specification of the environment.

Domain as a specification of the environment

$$\llbracket D \rrbracket = \{\sigma_e \mid \sigma_e \text{ complies with domain } D\}$$

where, an environment strategy σ_e complies with domain D , if for every trace $a_0, s_0, a_1, s_1, \dots, a_n, s_n$ of T_D (a_0 is the dummy *start* action) we have that $\sigma_e(a_0, \dots, a_{n+1}) = s'_{n+1}$ is such that $\delta(s_n, a_{n+1}, s_{n+1})$.

Planning in nondeterministic domains

Given an LTL_f task $Goal$ for the agent, and a domain D modeling the environment

- Realizability:

$$\text{check if } \exists \sigma_a. \forall \sigma_e \in \llbracket D \rrbracket. \text{trace}(\sigma_a, \sigma_e) \models Goal$$

- Synthesis:

$$\text{find } \sigma_a \text{ such that } \forall \sigma_e \in \llbracket D \rrbracket. \text{trace}(\sigma_a, \sigma_e) \models Goal$$

We can transfer the idea of working with a model of the world to synthesis

Synthesis for a task in an environment

Given a task $Task$ for the agent, and a specification Env of the possible environment strategies:

- Realizability:

check if $\exists \sigma_a. \forall \sigma_e \in \llbracket Env \rrbracket. trace(\sigma_a, \sigma_e) \models Task$

- Synthesis:

find σ_a such that $\forall \sigma_e \in \llbracket Env \rrbracket. trace(\sigma_a, \sigma_e) \models Task$

Specifying possible environment behaviors in LTL

- Can we use LTL/LTL_f to specify an environment, i.e., the possible environment strategies?
- Yes, through the notion of realizability!

Environment specifications in LTL

Let Env be an LTL/LTL_f formula over action and fluents.

$$\llbracket Env \rrbracket = \{ \sigma_e \mid \forall \sigma_a. trace(\sigma_a, \sigma_e) \models Env \}$$

i.e. Env denotes all environment behaviors that play according to the specification whatever is the agent behavior.

Consistent environment specifications

Is any LTL/LTL_f formula a valid environment specification? No, Env needs to be “consistent”!:

$$\llbracket Env \rrbracket \neq \emptyset \quad \text{i.e. } \exists \sigma_e. \forall \sigma_a. trace(\sigma_a, \sigma_e) \models Env$$

For example “eventually agent does action *dec*”

eventually dec

is not a valid specification of the environment, since the agent might decide not to do *dec*.

Specifying possible environment behaviors in LTL

- Can we use LTL/LTL_f to specify an environment, i.e., the possible environment strategies?
- Yes, through the notion of realizability!

Environment specifications in LTL

Let Env be an LTL/LTL_f formula over action and fluents.

$$\llbracket Env \rrbracket = \{\sigma_e \mid \forall \sigma_a. trace(\sigma_a, \sigma_e) \models Env\}$$

i.e. Env denotes all environment behaviors that play according to the specification whatever is the agent behavior.

Consistent environment specifications

Is any LTL/LTL_f formula a valid environment specification? No, Env needs to be “consistent”!:

$$\llbracket Env \rrbracket \neq \emptyset \quad \text{i.e. } \exists \sigma_e. \forall \sigma_a. trace(\sigma_a, \sigma_e) \models Env$$

For example “eventually agent does action dec ”

eventually dec

is not a valid specification of the environment, since the agent might decide not to do dec .

Specifying possible environment behaviors in LTL

- Can we use LTL/LTL_f to specify an environment, i.e., the possible environment strategies?
- Yes, through the notion of realizability!

Environment specifications in LTL

Let Env be an LTL/LTL_f formula over action and fluents.

$$\llbracket Env \rrbracket = \{ \sigma_e \mid \forall \sigma_a. trace(\sigma_a, \sigma_e) \models Env \}$$

i.e. Env denotes all environment behaviors that play according to the specification whatever is the agent behavior.

Consistent environment specifications

Is any LTL/LTL_f formula a valid environment specification? No, Env needs to be “consistent”!:

$$\llbracket Env \rrbracket \neq \emptyset \quad \text{i.e. } \exists \sigma_e. \forall \sigma_a. trace(\sigma_a, \sigma_e) \models Env$$

For example “eventually agent does action dec ”

eventually dec

is not a valid specification of the environment, since the agent might decide not to do dec .

Specifying possible environment behaviors in LTL

- Can we use LTL/LTL_f to specify an environment, i.e., the possible environment strategies?
- Yes, through the notion of realizability!

Environment specifications in LTL

Let Env be an LTL/LTL_f formula over action and fluents.

$$\llbracket Env \rrbracket = \{ \sigma_e \mid \forall \sigma_a. trace(\sigma_a, \sigma_e) \models Env \}$$

i.e. Env denotes all environment behaviors that play according to the specification whatever is the agent behavior.

Consistent environment specifications

Is any LTL/LTL_f formula a valid environment specification? No, Env needs to be “consistent”!:

$$\llbracket Env \rrbracket \neq \emptyset \quad \text{i.e. } \exists \sigma_e. \forall \sigma_a. trace(\sigma_a, \sigma_e) \models Env$$

For example “eventually agent does action dec ”

eventually dec

is not a valid specification of the environment, since the agent might decide not to do dec .

Check consistency of environment specifications

First we need to check **consistency** of the environment specification Env , i.e., our model of the world ($\llbracket Env \rrbracket \neq \emptyset$). We can do so by checking **unrealizability** of agent task Env (we exploit determinacy of resulting games):

$$\exists \sigma_a. \forall \sigma_e. \text{trace}(\sigma_a, \sigma_e) \models \neg Env$$

Theorem

Checking consistency of LTL/LTL_f environment specification is $2EXPTIME$ -complete.

Note: for LTL we could solve realizability for the environment directly.

Instead for LTL_f no, because the agent, and not the environment, has the choice of stopping the trace

– though we could solve a safety game instead of a reachability one (see later).

Solve synthesis

Theorem ([AminofDeGiacomoMuranoRubinICAPS2019])

Let $Task$ be a agent task and Env be a consistent LTL_f/LTL environment specification. Then

- There is **agent strategy realizing** $Task$ in Env iff there is an **agent strategy realizing** $Env \supset Task$

$$\exists \sigma_a. \forall \sigma_e \in \llbracket Env \rrbracket. trace(\sigma_a, \sigma_e) \models Task \text{ iff } \exists \sigma_a. \forall \sigma_e. trace(\sigma_a, \sigma_e) \models Env \supset Task$$

- Moreover, **every agent strategy realizing** $Env \supset Task$ is a **agent strategy realizing** $Task$ in Env

$$\text{for all } \sigma_a \text{ we have: } \forall \sigma_e. trace(\sigma_a, \sigma_e) \models Env \supset Task \text{ implies } \forall \sigma_e \in \llbracket Env \rrbracket. trace(\sigma_a, \sigma_e) \models Task$$

but not viceversa!

Hence, to find **agent strategy realizing** $Task$ under the environment specification Env , we can use standard LTL/LTL_f synthesis for

$$Env \supset Task$$

Theorem

Solving LTL/LTL_f synthesis under environment specification is 2EXPTIME-complete.

By the theorem above, the two problems

$$\exists \sigma_a. \forall \sigma_e \in \llbracket Env \rrbracket. \text{trace}(\sigma_a, \sigma_e) \models Task \quad \text{and} \quad \exists \sigma_a. \forall \sigma_e. \text{trace}(\sigma_a, \sigma_e) \models Env \supset Task$$

are **equi-realizable**, but **not equivalent** on strategy-by-strategy basis.

To see the latter consider the following counterexample:

Counterexample to equivalence

Let $Env \doteq f \supset a$ and $Task \doteq f \supset b$. Observe Env is a consistent environment specification (indeed, the environment can realize Env by playing $\neg f$ at the first step).

Moreover, every environment strategy realizing Env must begin by playing $\neg f$ (since otherwise the agent could play $\neg a$ on its first turn and falsify Env).

Thus, every agent strategy realizes $Task$ assuming Env (since the environment's first move is to play $\neg f$ which makes $Task$ true no matter what the agent does).

On the other hand, not every agent strategy realizes $Env \supset Task$ (indeed, the agent strategy which plays a on its first turn fails to satisfy the implication on the trace in which the environment plays f on its first turn).